WARSZTAT PRAKTYCZNY

SQL Server 2025 + Azure ML + OpenAI

Przewidywanie rezygnacji klientów klubu fitness i automatyczne maile retencyjne generowane z poziomu T-SQL

Parametr	Wartość
Cykl	Użycie AI w pracy z SQL Server
Scenariusz	Sieć klubów FitPoint (dane fikcyjne)
Poziom	zaawansowany SQL / T-SQL
Środowisko	SQL Server 2025 (17.x) Developer Edition, SSMS
Usługi zewnętrzne	Azure Machine Learning (endpoint /score), OpenAI API (chat completions)

Wprowadzenie

Cel warsztatu

Kursanci zbudują w SQL Server 2025 kompletny, sterowany z T-SQL proces retencji klientów:

1. Dane klientów są przechowywane w tabeli `dbo.Klienci`.
2. Procedura wysyła dane do modelu predykcyjnego w **Azure Machine Learning** i zapisuje prognozę w kolumnie `[Czy Zrezygnuje]`.
3. Dla każdego klienta z prognozą **Tak** procedura buduje prompt z danych wiersza, wysyła go do **OpenAI API** i zapisuje wygenerowany mail w kolumnie `TrescMaila`.
4. Całość uruchamia jedna procedura nadrzędna `EXEC dbo.usp_ProcesRetencji;`.

Architektura rozwiązania

Krok	Gdzie	Co się dzieje
1	SQL Server	Odczyt wierszy z `dbo.Klienci`, budowa JSON w formacie `input_data`
2	Azure ML	POST na `/score`, odpowiedź: tablica `["Nie","Tak",...]`
3	SQL Server	Parsowanie odpowiedzi (`OPENJSON`), UPDATE kolumny `[Czy Zrezygnuje]`
4	SQL Server	Dla wierszy z „Tak” podstawienie danych do szablonu promptu
5	OpenAI	POST na `/v1/chat/completions`, odpowiedź: treść maila
6	SQL Server	UPDATE kolumny `TrescMaila`, raport końcowy

> ⚠ **Status funkcji w SQL Server 2025**
>
> Procedura systemowa `sp_invoke_external_rest_endpoint` jest w SQL Server 2025 oznaczona w dokumentacji Microsoft Learn jako funkcja w wersji **preview** i jest domyślnie **wyłączona**. Przed zajęciami sprawdź aktualny status i składnię w dokumentacji (strona „sp_invoke_external_rest_endpoint (Transact-SQL)”).
>
> Wywołanie wysyła dane poza serwer. Na zajęciach używamy wyłącznie danych fikcyjnych, bez imion, nazwisk i adresów e-mail.

> ℹ **Wymagania wstępne**
>
> • SQL Server 2025 z uprawnieniami **sysadmin** (włączenie opcji serwera) oraz SSMS.
> • Wyjście HTTPS (port 443) z serwera SQL do `*.inference.ml.azure.com` oraz `api.openai.com`.
> • Klucz do endpointu Azure ML (`ws-fitpoint-ps`) oraz klucz OpenAI API z dostępnymi środkami.
> • Działający test z Postmana: dla trzech przykładowych wierszy odpowiedź `["Nie","Tak","Nie"]`.

MODUŁ 1

Baza FitPoint i tabela Klienci

Cel modułu

Utworzymy bazę danych i tabelę, której kolumny dokładnie odpowiadają kolumnom oczekiwanym przez model w Azure ML, plus dwie kolumny wynikowe.

Krok 1.1 – Baza danych

```
CREATE DATABASE FitPoint;
GO
USE FitPoint;
GO
```

Krok 1.2 – Tabela dbo.Klienci

Zwróć uwagę na dwie nazwy kolumn wymagające nawiasów kwadratowych: `[Index]` (słowo kluczowe T-SQL) oraz `[Czy Zrezygnuje]` (spacja w nazwie).

```
CREATE TABLE dbo.Klienci
(
    [Index]               INT           NOT NULL,
    StazMiesiace          INT           NOT NULL,
    WizytyMiesiecznie     INT           NOT NULL,
    DniOdOstatniejWizyty  INT           NOT NULL,
    TypKarnetu            NVARCHAR(20)  NOT NULL,
    OplataMiesieczna      DECIMAL(8,2)  NOT NULL,
    ZajeciaGrupowe        NVARCHAR(3)   NOT NULL,
    LiczbaReklamacji      INT           NOT NULL,
    Wiek                  INT           NOT NULL,
    [Czy Zrezygnuje]      NVARCHAR(3)   NULL,   -- wynik z Azure ML
    TrescMaila            NVARCHAR(MAX) NULL,   -- wynik z OpenAI
    CONSTRAINT PK_Klienci PRIMARY KEY ([Index]),
    CONSTRAINT CK_Klienci_ZajeciaGrupowe CHECK (ZajeciaGrupowe IN (N'Tak', N'Nie')),
    CONSTRAINT CK_Klienci_CzyZrezygnuje  CHECK ([Czy Zrezygnuje] IN (N'Tak', N'Nie'))
);
GO
```

> **Pytanie**
>
> Po co ograniczenie CHECK na kolumnie `[Czy Zrezygnuje]`, skoro wartość i tak przyjdzie z modelu? (Odpowiedź: to bezpiecznik. Jeśli model lub endpoint zwróci coś nieoczekiwanego, np. `"1"` albo komunikat błędu, UPDATE się nie powiedzie zamiast po cichu zapisać śmieci.)

Krok 1.3 – Dane fikcyjne

Pierwsze trzy wiersze to dokładnie te same dane, które testowaliśmy w Postmanie. Dzięki temu znamy oczekiwany wynik i możemy sprawdzić, czy integracja działa poprawnie.

```
INSERT INTO dbo.Klienci
    ([Index], StazMiesiace, WizytyMiesiecznie, DniOdOstatniejWizyty, TypKarnetu,
     OplataMiesieczna, ZajeciaGrupowe, LiczbaReklamacji, Wiek)
VALUES
    (0, 36, 12,  2, N'Open',     189.00, N'Tak', 0, 41),
```

```
    (1,  2,  1, 48, N'Student',   89.00, N'Nie', 2, 21),
    (2, 18,  8,  5, N'Standard', 129.00, N'Tak', 0, 35),
    (3,  4,  2, 30, N'Standard', 129.00, N'Nie', 1, 28),
    (4, 60, 15,  1, N'Open',     189.00, N'Tak', 0, 52),
    (5,  1,  0, 25, N'Student',   89.00, N'Nie', 3, 19),
    (6, 24,  6, 10, N'Standard', 129.00, N'Tak', 0, 44),
    (7,  6,  1, 40, N'Open',     189.00, N'Nie', 2, 33),
    (8, 12, 10,  3, N'Student',   89.00, N'Tak', 0, 23),
    (9,  3,  2, 21, N'Standard', 129.00, N'Nie', 1, 30);
GO
```

☑ Weryfikacja po module 1

☐ Tabela zawiera **10 wierszy**, a kolumny `[Czy Zrezygnuje]` i `TrescMaila` są puste we wszystkich wierszach.

☐ Kolumny i typy danych zgadzają się z poniższym zapytaniem (11 kolumn).

```
SELECT COUNT(*)                                     AS LiczbaWierszy,
       SUM(IIF([Czy Zrezygnuje] IS NULL, 1, 0))     AS BezPrognozy,
       SUM(IIF(TrescMaila IS NULL, 1, 0))           AS BezMaila
FROM dbo.Klienci;          -- oczekiwane: 10 | 10 | 10

SELECT c.column_id, c.name, t.name AS typ, c.max_length, c.is_nullable
FROM sys.columns AS c
JOIN sys.types   AS t ON t.user_type_id = c.user_type_id
WHERE c.object_id = OBJECT_ID(N'dbo.Klienci')
ORDER BY c.column_id;
```

☐ Próba wstawienia `ZajeciaGrupowe = N'Może'` kończy się błędem ograniczenia CHECK.

MODUŁ 2

Wywołania REST z T-SQL: konfiguracja i testy

Cel modułu

Przygotujemy serwer do wywoływania zewnętrznych API, bezpiecznie zapiszemy klucze i wykonamy po jednym ręcznym wywołaniu każdego API, zanim zamkniemy logikę w procedurach.

Krok 2.1 – Włączenie sp_invoke_external_rest_endpoint

```
EXECUTE sp_configure 'external rest endpoint enabled', 1;
RECONFIGURE WITH OVERRIDE;
GO
```

Uprawnienie do zmiany tej opcji mają role serwera **sysadmin** i **serveradmin**. Na serwerze współdzielonym zrobi to prowadzący.

Krok 2.2 – Klucz główny bazy (Database Master Key)

Poświadczenia na poziomie bazy (`DATABASE SCOPED CREDENTIAL`) są szyfrowane kluczem głównym bazy, więc musi on istnieć.

```
USE FitPoint;
GO
IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys
               WHERE name = N'##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = N'Zmien_To#Haslo_FitPoint2026!';
GO
```

Krok 2.3 – Poświadczenia z kluczami API

Klucz nigdy nie trafia do kodu procedury. Zapisujemy go w poświadczeniu typu `HTTPEndpointHeaders`, czyli jako nagłówek HTTP dołączany automatycznie do żądania. **Nazwa poświadczenia musi być adresem URL** (prefiksem adresu, który wywołujemy).

```
-- Azure Machine Learning (managed online endpoint): nagłówek Authorization: Bearer <klucz>
CREATE DATABASE SCOPED CREDENTIAL [https://ws-fitpoint-ps.polandcentral.inference.ml.azure.com]
WITH IDENTITY = 'HTTPEndpointHeaders',
     SECRET   = '{"Authorization":"Bearer <WKLEJ_KLUCZ_AZURE_ML>"}';
GO


-- OpenAI API
CREATE DATABASE SCOPED CREDENTIAL [https://api.openai.com]
WITH IDENTITY = 'HTTPEndpointHeaders',
     SECRET   = '{"Authorization":"Bearer <WKLEJ_KLUCZ_OPENAI>"}';
GO


-- Kontrola (sekretu nie da się odczytać – i dobrze)
SELECT name, credential_identity, create_date
FROM sys.database_scoped_credentials;
```

Uprawnienia dla użytkowników bez roli sysadmin

Wywołanie procedury wymaga uprawnienia bazodanowego `EXECUTE ANY EXTERNAL ENDPOINT`, a użycie poświadczenia uprawnienia `REFERENCES` do niego. Przykład dla roli aplikacyjnej:

```
CREATE ROLE RetencjaApp;
GRANT EXECUTE ANY EXTERNAL ENDPOINT TO RetencjaApp;
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::[https://api.openai.com] TO RetencjaApp;
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::[https://ws-fitpoint-
ps.polandcentral.inference.ml.azure.com] TO RetencjaApp;
```

Krok 2.4 – Ręczny test Azure ML (ten sam JSON co w Postmanie)

```
DECLARE @ret INT, @response NVARCHAR(MAX);
DECLARE @payload NVARCHAR(MAX) = N'{
  "input_data": {
    "columns": ["StazMiesiace","WizytyMiesiecznie","DniOdOstatniejWizyty","TypKarnetu",
                "OplataMiesieczna","ZajeciaGrupowe","LiczbaReklamacji","Wiek"],
    "index": [0, 1, 2],
    "data": [
      [36, 12, 2, "Open", 189.00, "Tak", 0, 41],
      [2, 1, 48, "Student", 89.00, "Nie", 2, 21],
      [18, 8, 5, "Standard", 129.00, "Tak", 0, 35]
    ]
  }
}';

EXEC @ret = sp_invoke_external_rest_endpoint
     @url        = N'https://ws-fitpoint-ps.polandcentral.inference.ml.azure.com/score',
     @method     = 'POST',
     @headers    = N'{"Content-Type":"application/json"}',
     @payload    = @payload,
     @credential = [https://ws-fitpoint-ps.polandcentral.inference.ml.azure.com],
     @timeout    = 60,
     @response   = @response OUTPUT;

SELECT @ret                                                  AS KodPowrotu,   -- 0 = HTTP 2xx
       JSON_VALUE(@response, '$.response.status.http.code') AS HttpStatus,
       JSON_QUERY(@response, '$.result')                     AS Wynik,
       @response                                             AS PelnaOdpowiedz;
```

Odpowiedź procedury to „koperta” JSON: w `$.response` są status i nagłówki HTTP, a w `$.result` właściwa treść zwrócona przez API. Pozycje tablicy w `$.result` (0, 1, 2) odpowiadają kolejności wierszy w `data`.

Krok 2.5 – Ręczny test OpenAI

Payload budujemy funkcjami `JSON_OBJECT` i `JSON_ARRAY`. Same zajmą się poprawnym escapowaniem cudzysłowów i polskich znaków, czego przy sklejaniu napisów łatwo zapomnieć.

```
DECLARE @ret INT, @response NVARCHAR(MAX);
DECLARE @payload NVARCHAR(MAX) = JSON_OBJECT(
    'model'    : N'gpt-4o-mini',
    'messages' : JSON_ARRAY(
                    JSON_OBJECT('role': 'user',
                                'content': N'Napisz jedno zdanie powitania dla klubu fitness.')
                 )
);

EXEC @ret = sp_invoke_external_rest_endpoint
     @url        = N'https://api.openai.com/v1/chat/completions',
     @method     = 'POST',
     @headers    = N'{"Content-Type":"application/json"}',
     @payload    = @payload,
     @credential = [https://api.openai.com],
     @timeout    = 120,
```

```
    @response   = @response OUTPUT;

SELECT @ret                                                          AS KodPowrotu,
       JSON_VALUE(@response, '$.response.status.http.code')          AS HttpStatus,
       JSON_VALUE(@response, '$.result.choices[0].message.content') AS Tresc,
       JSON_VALUE(@response, '$.result.usage.total_tokens')         AS Tokeny;
```

⚠ Nazwa modelu OpenAI

`gpt-4o-mini` to przykład. Listę aktualnie dostępnych modeli sprawdź w dokumentacji OpenAI lub na swoim koncie przed zajęciami. Niektóre nowsze modele nie akceptują parametrów takich jak `temperature` czy `max_tokens`, dlatego w warsztacie ich nie ustawiamy.

☑ Weryfikacja po module 2

- ☐ Opcja serwera jest włączona: `SELECT name, value_in_use FROM sys.configurations WHERE name = N'external rest endpoint enabled';` zwraca **1**.
- ☐ W `sys.database_scoped_credentials` widać **dwa** poświadczenia z `credential_identity = HTTPEndpointHeaders`.
- ☐ Test Azure ML: `KodPowrotu = 0`, `HttpStatus = 200`, `Wynik = ["Nie","Tak","Nie"]`, czyli **identycznie jak w Postmanie**.
- ☐ Test OpenAI: `HttpStatus = 200` i sensowne zdanie po polsku w kolumnie `Tresc`.
- ☐ Jeśli którykolwiek punkt nie przechodzi, zajrzyj do tabeli „Typowe problemy” na końcu materiału, zanim przejdziesz dalej.

MODUŁ 3

Procedura scoringu w Azure ML

Cel modułu

Zamieniamy ręczny test w procedurę `dbo.usp_PrzewidzRezygnacje`, która sama zbuduje JSON z tabeli, wyśle go do modelu i zapisze odpowiedź w kolumnie `[Czy Zrezygnuje]`.

Kluczowy problem: dopasowanie odpowiedzi do wierszy

Model zwraca **samą tablicę** wartości, bez identyfikatorów klientów. Jedyną informacją łączącą wynik z wierszem jest **pozycja** w tablicy. Dlatego:

- ustalamy kolejność wierszy raz (`ROW_NUMBER() ... ORDER BY [Index]`) i zapisujemy ją jako kolumnę `Pozycja` w tabeli tymczasowej,
- budując JSON, sortujemy `STRING_AGG ... WITHIN GROUP (ORDER BY Pozycja)`,
- parsując odpowiedź, łączymy `OPENJSON(...).[key]` (indeks w tablicy od 0) z `Pozycja`,
- sprawdzamy, czy liczba wyników równa się liczbie wysłanych wierszy. Jeśli nie, przerywamy bez zapisu.

Krok 3.1 – Budowa JSON z tabeli (najpierw poza procedurą)

Uruchom fragment samodzielnie i porównaj wynik z JSON-em z Postmana.

```
DECLARE @index NVARCHAR(MAX), @data NVARCHAR(MAX), @payload NVARCHAR(MAX);

SELECT
    @index = N'[' + STRING_AGG(CAST([Index] AS NVARCHAR(MAX)), N',')
                    WITHIN GROUP (ORDER BY [Index]) + N']',
    @data  = N'[' + STRING_AGG(CAST(JSON_ARRAY(StazMiesiace, WizytyMiesiecznie,
                                               DniOdOstatniejWizyty, TypKarnetu,
                                               OplataMiesieczna, ZajeciaGrupowe,
                                               LiczbaReklamacji, Wiek) AS NVARCHAR(MAX)), N',')
                    WITHIN GROUP (ORDER BY [Index]) + N']'
FROM dbo.Klienci;

SET @payload = JSON_OBJECT(
    'input_data': JSON_OBJECT(
        'columns': JSON_ARRAY(N'StazMiesiace', N'WizytyMiesiecznie', N'DniOdOstatniejWizyty',
                              N'TypKarnetu', N'OplataMiesieczna', N'ZajeciaGrupowe',
                              N'LiczbaReklamacji', N'Wiek'),
        'index'   : JSON_QUERY(@index),
        'data'    : JSON_QUERY(@data)
    )
);

SELECT @payload AS Payload, ISJSON(@payload) AS CzyPoprawnyJSON;
```

`JSON_ARRAY` zachowuje typy: liczby zostają liczbami, napisy dostają cudzysłowy. `JSON_QUERY` mówi funkcji `JSON_OBJECT`, że przekazywany napis jest już gotowym fragmentem JSON i nie należy go escapować.

Krok 3.2 – Procedura dbo.usp_PrzewidzRezygnacje

```
CREATE OR ALTER PROCEDURE dbo.usp_PrzewidzRezygnacje
    @TylkoBezPrognozy BIT = 0          -- 1 = oceniaj tylko wiersze z pustą prognozą
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    DECLARE @url      NVARCHAR(4000) = N'https://ws-fitpoint-
ps.polandcentral.inference.ml.azure.com/score',
            @index    NVARCHAR(MAX),
            @data     NVARCHAR(MAX),
            @payload  NVARCHAR(MAX),
            @response NVARCHAR(MAX),
            @ret      INT,
            @wyslane  INT,
            @odebrane INT,
            @msg      NVARCHAR(2048);

    /* 1. Wiersze do oceny w ustalonej kolejności */
    DROP TABLE IF EXISTS #DoOceny;

    SELECT [Index], StazMiesiace, WizytyMiesiecznie, DniOdOstatniejWizyty, TypKarnetu,
           OplataMiesieczna, ZajeciaGrupowe, LiczbaReklamacji, Wiek,
           ROW_NUMBER() OVER (ORDER BY [Index]) - 1 AS Pozycja      -- 0, 1, 2, ...
    INTO #DoOceny
    FROM dbo.Klienci
    WHERE @TylkoBezPrognozy = 0 OR [Czy Zrezygnuje] IS NULL;

    SET @wyslane = @@ROWCOUNT;
    IF @wyslane = 0
    BEGIN
        PRINT N'Brak klientów do oceny.';
        RETURN;
    END;

    /* 2. Budowa żądania w formacie input_data */
    SELECT
        @index = N'[' + STRING_AGG(CAST([Index] AS NVARCHAR(MAX)), N',')
                        WITHIN GROUP (ORDER BY Pozycja) + N']',
        @data  = N'[' + STRING_AGG(CAST(JSON_ARRAY(StazMiesiace, WizytyMiesiecznie,
                                                   DniOdOstatniejWizyty, TypKarnetu,
                                                   OplataMiesieczna, ZajeciaGrupowe,
                                                   LiczbaReklamacji, Wiek) AS NVARCHAR(MAX)), N',')
                        WITHIN GROUP (ORDER BY Pozycja) + N']'
    FROM #DoOceny;

    SET @payload = JSON_OBJECT(
        'input_data': JSON_OBJECT(
            'columns': JSON_ARRAY(N'StazMiesiace', N'WizytyMiesiecznie', N'DniOdOstatniejWizyty',
                                  N'TypKarnetu', N'OplataMiesieczna', N'ZajeciaGrupowe',
                                  N'LiczbaReklamacji', N'Wiek'),
            'index'  : JSON_QUERY(@index),
            'data'   : JSON_QUERY(@data)
        )
    );

    /* 3. Wywołanie modelu */
    EXEC @ret = sp_invoke_external_rest_endpoint
         @url        = @url,
         @method     = 'POST',
         @headers    = N'{"Content-Type":"application/json"}',
         @payload    = @payload,
         @credential = [https://ws-fitpoint-ps.polandcentral.inference.ml.azure.com],
         @timeout    = 120,
         @response   = @response OUTPUT;

    IF @ret <> 0
    BEGIN
        SET @msg = CONCAT(N'Azure ML zwrócił błąd HTTP ', @ret, N'. Odpowiedź: ', LEFT(@response, 1500));
        THROW 50001, @msg, 1;
    END;
```

```
    /* 4. Kontrola spójności odpowiedzi */
    SELECT @odebrane = COUNT(*) FROM OPENJSON(@response, '$.result');

    IF @odebrane <> @wyslane
    BEGIN
        SET @msg = CONCAT(N'Wysłano ', @wyslane, N' wierszy, odebrano ', @odebrane,
                          N' prognoz. Zapis przerwany.');
        THROW 50002, @msg, 1;
    END;

    /* 5. Zapis prognoz (dopasowanie po pozycji w tablicy) */
    UPDATE k
    SET    k.[Czy Zrezygnuje] = w.[value],
           -- klient, który nie jest już zagrożony, nie potrzebuje maila
           k.TrescMaila = IIF(w.[value] = N'Nie', NULL, k.TrescMaila)
    FROM   dbo.Klienci           AS k
    JOIN   #DoOceny              AS d ON d.[Index] = k.[Index]
    JOIN   OPENJSON(@response, '$.result') AS w ON CAST(w.[key] AS INT) = d.Pozycja;

    /* 6. Podsumowanie */
    SELECT [Czy Zrezygnuje], COUNT(*) AS LiczbaKlientow
    FROM   dbo.Klienci
    GROUP BY [Czy Zrezygnuje];
END;
GO
```

Krok 3.3 – Uruchomienie

```
EXEC dbo.usp_PrzewidzRezygnacje;

SELECT [Index], StazMiesiace, WizytyMiesiecznie, DniOdOstatniejWizyty, [Czy Zrezygnuje]
FROM dbo.Klienci
ORDER BY [Index];
```

Dyskusja: czy model ma rację?

Zanim zaufamy prognozom, spójrz na dane. Czy klienci z prognozą „Tak” rzeczywiście mają krótki staż, mało wizyt, dawno nie byli i składali reklamacje? Model to narzędzie statystyczne: może się mylić, a odpowiedzialność za decyzję biznesową (np. rabat 50%) ponosi człowiek.

Weryfikacja po module 3

- ☐ Procedura kończy się bez błędu, a podsumowanie nie zawiera wiersza z `[Czy Zrezygnuje] = NULL`.
- ☐ Wiersze 0, 1 i 2 mają prognozy **Nie, Tak, Nie**, czyli zgodne z testem w Postmanie (to nasz „złoty standard”).
- ☐ Ponowne `EXEC dbo.usp_PrzewidzRezygnacje @TylkoBezPrognozy = 1;` wypisuje „Brak klientów do oceny.”
- ☐ Test negatywny: zmień w procedurze nazwę kolumny w `columns` (np. `Wiek` → `WiekKlienta`), uruchom i sprawdź, że dostajesz czytelny błąd 50001, a dane w tabeli **nie zmieniły się**. Przywróć poprawną nazwę.

MODUŁ 4

Procedura generowania maili w OpenAI

Cel modułu

Dla każdego klienta z prognozą **Tak** budujemy prompt z danych wiersza, wysyłamy go do OpenAI i zapisujemy wynik w kolumnie `TrescMaila`.

Szablon promptu

Szablon od działu marketingu (w nawiasach kwadratowych nazwy kolumn):

> **Prompt**
>
> Biorąc pod uwagę, że klient był z nami **[StazMiesiace]** i odwiedzał nas **[WizytyMiesiecznie]**, a ostatni raz odwiedził nas **[DniOdOstatniejWizyty]** dni temu, wygeneruj mail w 3-4 zdaniach próbujący przekonać do powrotu do klubu Fitness proponując mu 50% z **[OplataMiesieczna]**.

Podczas podstawiania dodajemy jednostki („miesięcy”, „razy w miesiącu”, „zł”), bo sama liczba bez kontekstu bywa przez model źle interpretowana. Dodatkowo wiadomość systemowa (`role: system`) ustala styl i format odpowiedzi.

Krok 4.1 – Podgląd promptów (bez wywoływania API)

Dobra praktyka: najpierw zobacz, co dokładnie wyślesz. To nic nie kosztuje i pozwala wyłapać błędy w podstawianiu.

```
SELECT [Index],
       CONCAT(N'Biorąc pod uwagę, że klient był z nami ', StazMiesiace, N' miesięcy',
              N' i odwiedzał nas ', WizytyMiesiecznie, N' razy w miesiącu,',
              N' a ostatni raz odwiedził nas ', DniOdOstatniejWizyty, N' dni temu,',
              N' wygeneruj mail w 3-4 zdaniach próbujący przekonać do powrotu do klubu Fitness,',
              N' proponując mu 50% z ', OplataMiesieczna, N' zł.') AS Prompt
FROM dbo.Klienci
WHERE [Czy Zrezygnuje] = N'Tak'
ORDER BY [Index];
```

Krok 4.2 – Procedura dbo.usp_GenerujMaileRetencyjne

Wywołujemy API osobno dla każdego klienta, bo każdy mail jest inny. Błąd przy jednym kliencie nie zatrzymuje pozostałych: zostaje zalogowany, a kolumna `TrescMaila` pozostaje pusta, więc kolejne uruchomienie spróbuje ponownie.

```
CREATE OR ALTER PROCEDURE dbo.usp_GenerujMaileRetencyjne
    @Model    NVARCHAR(100) = N'gpt-4o-mini',
    @Nadpisz  BIT = 0                    -- 1 = wygeneruj ponownie również istniejące maile
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @url      NVARCHAR(4000) = N'https://api.openai.com/v1/chat/completions',
            @system   NVARCHAR(MAX)  = N'Jesteś copywriterem klubu fitness FitPoint. '
                                     + N'Piszesz po polsku, ciepło i konkretnie. '
```

```
                                + N'Zwracasz wyłącznie treść maila (bez tematu, bez komentarzy), '
                                + N'zakończoną podpisem: Zespół FitPoint.',
        @idx      INT,
        @prompt   NVARCHAR(MAX),
        @payload  NVARCHAR(MAX),
        @response NVARCHAR(MAX),
        @mail     NVARCHAR(MAX),
        @ret      INT,
        @ok       INT = 0,
        @bledy    INT = 0;

DECLARE kur CURSOR LOCAL FAST_FORWARD FOR
    SELECT [Index]
    FROM   dbo.Klienci
    WHERE  [Czy Zrezygnuje] = N'Tak'
      AND  (@Nadpisz = 1 OR TrescMaila IS NULL)
    ORDER BY [Index];

OPEN kur;
FETCH NEXT FROM kur INTO @idx;

WHILE @@FETCH_STATUS = 0
BEGIN
    BEGIN TRY
        /* 1. Podstawienie danych do szablonu */
        SELECT @prompt = CONCAT(
                   N'Biorąc pod uwagę, że klient był z nami ', StazMiesiace, N' miesięcy',
                   N' i odwiedzał nas ', WizytyMiesiecznie, N' razy w miesiącu,',
                   N' a ostatni raz odwiedził nas ', DniOdOstatniejWizyty, N' dni temu,',
                   N' wygeneruj mail w 3-4 zdaniach próbujący przekonać do powrotu do klubu Fitness,',
                   N' proponując mu 50% z ', OplataMiesieczna, N' zł.')
        FROM   dbo.Klienci
        WHERE  [Index] = @idx;

        /* 2. Payload chat completions */
        SET @payload = JSON_OBJECT(
            'model'   : @Model,
            'messages': JSON_ARRAY(
                            JSON_OBJECT('role': 'system', 'content': @system),
                            JSON_OBJECT('role': 'user',   'content': @prompt)
                        )
        );

        /* 3. Wywołanie OpenAI */
        SET @response = NULL;
        EXEC @ret = sp_invoke_external_rest_endpoint
             @url        = @url,
             @method     = 'POST',
             @headers    = N'{"Content-Type":"application/json"}',
             @payload    = @payload,
             @credential = [https://api.openai.com],
             @timeout    = 120,
             @response   = @response OUTPUT;

        IF @ret = 0
        BEGIN
            /* 4. Wyciągnięcie treści (OPENJSON + NVARCHAR(MAX) – bez limitu 4000 znaków) */
            SELECT @mail = Tresc
            FROM   OPENJSON(@response, '$.result.choices')
                   WITH (Tresc NVARCHAR(MAX) '$.message.content');

            UPDATE dbo.Klienci
            SET    TrescMaila = LTRIM(RTRIM(@mail))
            WHERE  [Index] = @idx;

            SET @ok += 1;
        END
        ELSE
        BEGIN
            SET @bledy += 1;
            PRINT CONCAT(N'Klient ', @idx, N': HTTP ', @ret, N' – ',
                         JSON_VALUE(@response, '$.result.error.message'));
        END;
    END TRY
    BEGIN CATCH
```

```
            SET @bledy += 1;
            PRINT CONCAT(N'Klient ', @idx, N': ', ERROR_MESSAGE());
        END CATCH;

        WAITFOR DELAY '00:00:01';   -- delikatnie wobec limitów zapytań (rate limit)
        FETCH NEXT FROM kur INTO @idx;
    END;

    CLOSE kur;
    DEALLOCATE kur;

    SELECT @ok AS WygenerowaneMaile, @bledy AS Bledy;
END;
GO
```

Krok 4.3 – Uruchomienie i przegląd

```
EXEC dbo.usp_GenerujMaileRetencyjne;

SELECT [Index], StazMiesiace, OplataMiesieczna, [Czy Zrezygnuje], TrescMaila
FROM dbo.Klienci
ORDER BY [Index];
```

Dlaczego kursor, a nie jedno zapytanie?

`sp_invoke_external_rest_endpoint` to procedura składowana, więc nie da się jej wywołać w `SELECT` ani `UPDATE` dla wielu wierszy naraz. Przy kilkunastu klientach kursor jest czytelny i wystarczający. Przy tysiącach rozważ przetwarzanie partiami, kolejkę albo zewnętrzny proces.

Weryfikacja po module 4

- ☐ Wynik procedury: `WygenerowaneMaile` = liczba klientów z prognozą „Tak”, `Bledy = 0`.
- ☐ Każdy wiersz z „Tak” ma niepusty `TrescMaila`, a każdy wiersz z „Nie” ma `TrescMaila = NULL`:

```
SELECT [Czy Zrezygnuje],
       SUM(IIF(TrescMaila IS NULL, 0, 1)) AS ZMailem,
       SUM(IIF(TrescMaila IS NULL, 1, 0)) AS BezMaila
FROM dbo.Klienci
GROUP BY [Czy Zrezygnuje];
```

- ☐ Przeczytaj maile: czy mają 3–4 zdania, są po polsku i zawierają ofertę 50%?
- ☐ Ponowne uruchomienie bez `@Nadpisz = 1` nie wywołuje API (wynik 0 / 0), więc nie płacimy dwa razy za to samo.

MODUŁ 5

Procedura nadrzędna i krytyczna weryfikacja

Krok 5.1 – Jedna procedura do uruchamiania całego procesu

```
CREATE OR ALTER PROCEDURE dbo.usp_ProcesRetencji
    @TylkoNowi BIT = 0
AS
BEGIN
    SET NOCOUNT ON;

    PRINT N'Krok 1/2: prognoza rezygnacji (Azure ML)...';
    EXEC dbo.usp_PrzewidzRezygnacje @TylkoBezPrognozy = @TylkoNowi;

    PRINT N'Krok 2/2: maile retencyjne (OpenAI)...';
    EXEC dbo.usp_GenerujMaileRetencyjne;

    SELECT [Index], [Czy Zrezygnuje],
           LEFT(TrescMaila, 120) + IIF(LEN(TrescMaila) > 120, N'...', N'') AS PoczatekMaila
    FROM dbo.Klienci
    ORDER BY [Index];
END;
GO
```

Krok 5.2 – Test end-to-end od zera

```
-- reset wyników
UPDATE dbo.Klienci SET [Czy Zrezygnuje] = NULL, TrescMaila = NULL;

-- pełny proces
EXEC dbo.usp_ProcesRetencji;

-- nowy klient dopisany po czasie: oceniamy tylko jego
INSERT INTO dbo.Klienci ([Index], StazMiesiace, WizytyMiesiecznie, DniOdOstatniejWizyty,
                         TypKarnetu, OplataMiesieczna, ZajeciaGrupowe, LiczbaReklamacji, Wiek)
VALUES (10, 3, 1, 35, N'Standard', 129.00, N'Nie', 2, 26);

EXEC dbo.usp_ProcesRetencji @TylkoNowi = 1;
```

Krok 5.3 – Krytyczna weryfikacja treści wygenerowanych przez AI

Model językowy może policzyć rabat niepoprawnie, dopisać obietnice, których klub nie złożył, albo zignorować limit zdań. Sprawdzamy to zapytaniem, a nie „na oko”.

```
SELECT [Index],
       OplataMiesieczna,
       CAST(OplataMiesieczna * 0.5 AS DECIMAL(8,2))                    AS CenaPoRabacie,
       IIF(TrescMaila LIKE N'%50[%]%' OR TrescMaila LIKE N'%50 [%]%'
           OR TrescMaila LIKE N'%połow%', N'OK', N'SPRAWDŹ')            AS CzyJestRabat,
       LEN(TrescMaila) - LEN(REPLACE(REPLACE(REPLACE(TrescMaila, N'.', N''),
                                     N'!', N''), N'?', N''))            AS PrzyblizonaLiczbaZdan,
       TrescMaila
FROM dbo.Klienci
WHERE [Czy Zrezygnuje] = N'Tak'
ORDER BY [Index];
```

Jeśli mail podaje kwotę po rabacie, porównaj ją z kolumną `CenaPoRabacie`. Liczba zdań jest tylko przybliżeniem (np. „zł.” albo skróty zawyżą wynik), ale wystarczy, by wskazać maile do ręcznego przejrzenia.

Bezpieczeństwo i odpowiedzialność

Ryzyko	Jak je ograniczamy w tym warsztacie
Wyciek danych osobowych (RODO)	Do API trafiają tylko cechy liczbowe i kategorie. Brak imion, nazwisk, e-maili i numerów karnetów.
Klucze API w kodzie	Klucze tylko w `DATABASE SCOPED CREDENTIAL`, zaszyfrowane kluczem głównym bazy, nieczytelne z widoków systemowych.
Nadmierne uprawnienia	`EXECUTE ANY EXTERNAL ENDPOINT` tylko dla dedykowanej roli. Funkcja serwera włączona świadomie.
Eksfiltracja danych	Każdy, kto może wywołać tę procedurę systemową, może wysłać dane pod dowolny adres HTTPS. Audytuj, kto ma uprawnienie, i kontroluj ruch wychodzący (firewall).
Błędy i halucynacje AI	Mail nie jest wysyłany automatycznie. Kolumna `TrescMaila` to szkic do akceptacji przez człowieka.
Koszty	Generujemy tylko brakujące maile (`@Nadpisz = 0`). Zużycie tokenów widać w `$.result.usage`.

Weryfikacja końcowa

- ☐ Po resecie i `EXEC dbo.usp_ProcesRetencji;` wszystkie wiersze mają prognozę, a wiersze 0–2 nadal dają **Nie, Tak, Nie**.
- ☐ Nowy klient (Index = 10) został oceniony bez ponownego wysyłania pozostałych 10 wierszy (sprawdź komunikaty w zakładce Messages).
- ☐ Zapytanie kontrolne z kroku 5.3 nie pokazuje żadnego `SPRAWDŹ` albo umiesz wyjaśnić, dlaczego się pojawił.
- ☐ Potrafisz wskazać, gdzie w rozwiązaniu jest klucz API i dlaczego nie ma go w kodzie procedur.
- ☐ Potrafisz wyjaśnić, jak procedura dopasowuje odpowiedź modelu do konkretnego klienta.

Typowe problemy i rozwiązania

Objaw	Prawdopodobna przyczyna	Rozwiązanie
Błąd, że funkcja jest wyłączona	Opcja serwera nie jest włączona	`sp_configure 'external rest endpoint enabled', 1; RECONFIGURE WITH OVERRIDE;`
Błąd tworzenia poświadczenia	Brak klucza głównego bazy	Krok 2.2: `CREATE MASTER KEY ...`
Poświadczenie „nie pasuje” do URL	Nazwa poświadczenia nie jest prefiksem wywoływanego adresu (literówka, inny host)	Porównaj nazwę z `sys.database_scoped_credentials` z `@url`
HTTP 401	Zły klucz albo brak słowa `Bearer` w nagłówku	Odtwórz poświadczenie: `DROP` + `CREATE`, sprawdź JSON w `SECRET`
HTTP 400 / 424 / 500 z Azure ML	Inne nazwy kolumn, kolejność lub typy niż oczekuje model	Porównaj `@payload` z JSON-em z Postmana (krok 3.1)
`$.result` jest napisem, a nie tablicą	Skrypt scoringowy zwraca JSON jako tekst (podwójne kodowanie)	`OPENJSON(JSON_VALUE(@response, '$.result'))` zamiast `OPENJSON(@response, '$.result')`
HTTP 429 z OpenAI	Limit zapytań lub brak środków na koncie	Zwiększ `WAITFOR DELAY`, sprawdź limity i saldo konta
HTTP 404 / model not found	Nieaktualna nazwa modelu	Zmień `@Model` na model dostępny na koncie
Błędy TLS / połączenia	Firewall, proxy lub brak zaufania do certyfikatu	Sprawdź ruch wychodzący 443 i magazyn certyfikatów serwera
Puste `TrescMaila` mimo HTTP 200	Inna struktura odpowiedzi API	Obejrzyj pełne `@response` i popraw ścieżkę JSON

Zadania dodatkowe

1. Dodaj tabelę `dbo.LogWywolanAPI` (czas, API, status HTTP, liczba tokenów, czas trwania) i zapisuj w niej każde wywołanie.
2. Przerób `usp_PrzewidzRezygnacje` tak, aby wysyłała dane partiami po N wierszy (parametr `@RozmiarPartii`).
3. Dodaj kolumnę `TematMaila` i poproś model o zwrócenie odpowiedzi w JSON (`{"temat": ..., "tresc": ...}`). Jak zweryfikować, że model faktycznie zwrócił poprawny JSON? (Podpowiedź: `ISJSON`.)
4. Uzależnij wysokość rabatu od typu karnetu (np. Student 30%, pozostałe 50%) i przenieś tę regułę do tabeli konfiguracyjnej, a nie do promptu.
5. Zamień OpenAI na Azure OpenAI: zmień URL, poświadczenie (nagłówek `api-key`) i porównaj strukturę odpowiedzi.